

DIGITAfrica: Summer School on Digital Technologies

**Configuration and System Management with
Ansible**

Ansible Intro

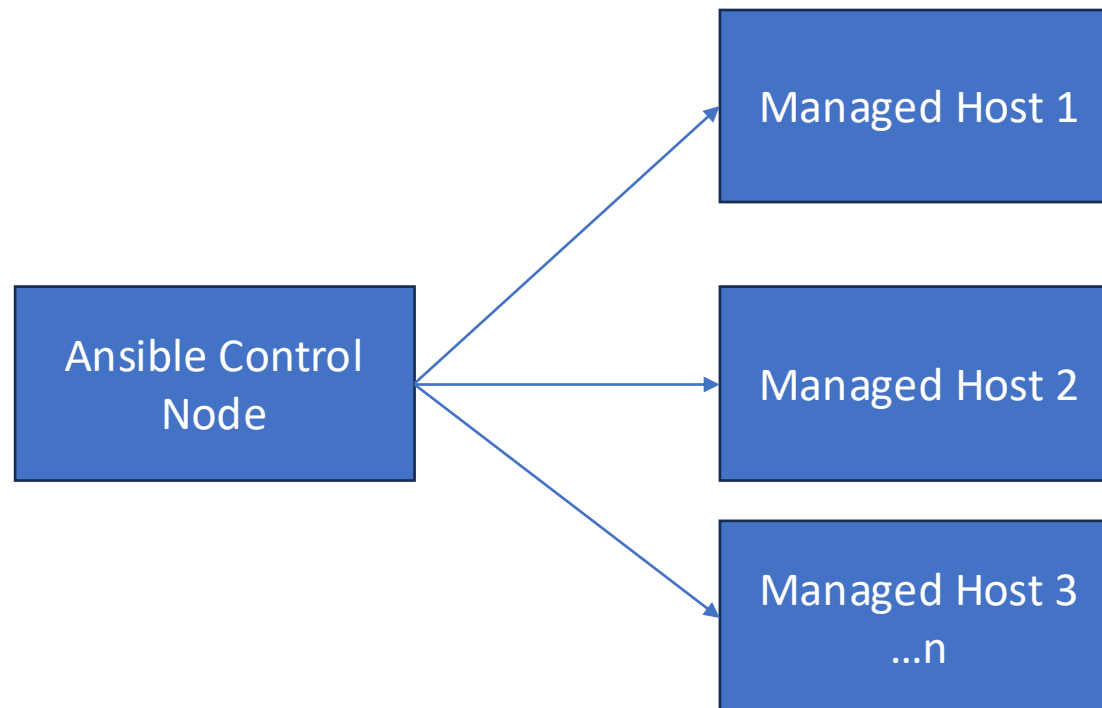
- Ansible is described as a configuration management tool.
- Config management means writing some kind of state description that ensures servers are in the desired state where: the right packages are installed, configuration files have the expected values and have the expected permissions, and the right services are running
- Deployment means generating binaries or static assets from source code, copying the required files to servers, configuring properties and environment variables, and starting services in the correct order.
- Orchestration, on the other hand, is the process of coordinating deployment when multiple remote servers are involved, and things must happen in a specific order
- Looking at this- Ansible can do both configuration management and Deployment and orchestration of services

Why Ansible?

- *Ansible is an open-source IT automation tool that lets you describe your infrastructure as code — and apply it repeatedly, reliably, and safely.*
- **Agentless**- No software needed on target machines — uses SSH to communicate.
- **Human-Readable**- Written in YAML — easy to read, write, and review as a team.
- **Idempotent** -Running a playbook multiple times produces the same result safely.
- **Modular** -Built-in modules handle everything: packages, files, services, cloud.

How Ansible Works

- Ansible connects to remote host(s) via SSH and executes configuration, deployment, and orchestration management instructions (tasks) using a script called an Ansible playbook.
- The interaction is usually an Ansible Control Host and Managed host(s)



How Ansible Works. Example Task

```
- name: Install nginx
  package:
    name: nginx
```

Ansible will do the following:

1. Generate a Python script that installs the NGINX package
2. Copy the script to host 1, host 2, host 3 Host n
3. Execute the script on host 1, host 2, host 3 Host n
4. Wait for the script to complete execution on all hosts

Up and Running with Ansible: What you need to do

- Ansible is agentless- it does not need an agent on the target host to work
- Ansible is written in python so it only expects that the Linux/macOS/BSD systems have Python installed
- You need to have SSH access and root privileges on the managed host(s)
- You only need to install Ansible on the Ansible Control Host
- Install using your Linux/macOS package managers or using the pip Python package manager
- **Note: Windows is not officially supported to run Ansible, but you can fully manage Windows systems remotely with Ansible, with PowerShell over WinRM under the hood. Or you can go the WSL (Windows Subsystem for Linux) route**

Lab 1: Install and Get Started

- <https://github.com/HumpreyDev/Strathmore-University-Summer-School-on-Digital-Technologies/tree/main> → Go to Lab 1 Directory

Ansible Projects

- Ansible recommends setting up your project in a modular fashion. This makes it more scalable, easier to manage and allows seamless collaboration. This structure makes the separation of tasks and variables, the reuse of project components such as roles (we will demonstrate this later) also maintains idempotency and readability.
 - Idempotency is key to Ansible's Infrastructure as code (IaC) management. This is the ability of an operation to produce the same result regardless of how many times it is repeated

Ansible Playbook

- A Playbook is the term that Ansible uses for a configuration management script.
- Playbooks are YAML files that inform Ansible about what it should do. Ansible expects to work its way through a list of commands and arguments, as well as any configuration details that may be required
- The command: `ansible-playbook {name_of_playbook.yaml}` runs your playbook
- <https://github.com/HumpreyDev/Strathmore-University-Summer-School-on-Digital-Technologies/tree/main> → . Go to Lab 2.0 Directory to run first ping playbook

Lab 2.0 Ping Playbook > ! ping_playbook.yaml

```
1  ---
2
3  - hosts: 192.168.64.6
4    tasks:
5      - name: Ping the target host to check connectivity
6        ping:
7
```

Basic Syntax rules when working with YAML and Ansible

- **File Structure:** Start every YAML file with `---` to indicate the beginning of the document.
- **Indentation and Spacing:** Use two spaces per indentation level. Ensure proper spacing within lists and around colons
- **Case Sensitivity:** YAML is case-sensitive; maintain consistent casing for keys
- Save files with `.yaml` or `.yml`
- **Variable Spacing:** Always ensure there is a space inside the curly braces when using variables, e.g., `{{ variable }}`, not `{{variable}}`.

A more complex playbook

- The playbook in <https://github.com/HumpreyDev/Strathmore-University-Summer-School-on-Digital-Technologies/tree/main> → Go to Lab 2.1:All-In-One-Project Directory

1. Installs apache web server.
2. Creates the first web page content
3. Installs Firewall and enables port 80
4. Tests web server locally

What's wrong with this playbook?

Ansible Projects File Structure

```
.
├── ansible.cfg
├── inventory
│   └── hosts.yaml
├── playbooks
│   └── sample_playbook.yaml
├── roles
│   └── sample_service
│       └── tasks
```

- Inventory: This defines the target hosts, at times, a group of hosts. The files are either saved as .ini or .yaml format.
- Playbooks- entry point to automation execution files and have the playbook files that will be run
- Roles - are reusable units of automation a service or a task. Each role usually has a separate directory for different tasks
- ansible.cfg is Ansible's **configuration file**. It controls how Ansible behaves when running playbooks, and lets you set default settings so you don't have to repeat them on every command or in every playbook.

A more complex playbook v2

- The playbook in <https://github.com/HumpreyDev/Strathmore-University-Summer-School-on-Digital-Technologies/tree/main> → Go to Lab 2.2 Ansible-Directories-Project Directory

We fix the previous all-in-one project into the desired Ansible project file structure. The playbook still does the following:

1. Installs Apache web server.
2. Creates the first web page content
3. Installs Firewalld and enables port 80
4. Tests web server locally

What's good with this organisation?

Another way of organizing Ansible Projects

File Structure

- The playbook in <https://github.com/HumpreyDev/Strathmore-University-Summer-School-on-Digital-Technologies/tree/main> → Go to Lab 2.3 Ansible-Directories-Project-S2 Directory

```
.
├── inventory
│   └── hosts
│       └── hosts
├── nginx_playbook.yaml
├── roles
│   └── nginx_server
│       └── tasks
```

- What do you notice that is different?
- What do you think can be moved to the ansible.cfg file?

Ansible Modules Used in This Playbook: Ref Github: Lab 3.1

- *A module is a unit of work. Ansible ships with thousands of built-in modules.*

apt

Manages Debian/Ubuntu packages.
Install, remove, or update with state: latest/present/absent.

apt_key

Adds a GPG key to the system keyring.
Ensures packages are from a trusted, verified source.

apt_repository

Adds an external software repository.
Tells apt where to find third-party packages like Docker.

pip

Installs Python packages via pip.
Used to install the Docker SDK for Python.

command

Runs a raw shell command.
No shell parsing — safer than shell: for simple commands.

debug

Prints messages or variable values.
Essential for verifying output during a playbook run.

Package Management with the apt Module

Installing a single package:

```
single_package.yml

- name: Install aptitude
  apt:
    name: aptitude
    state: latest
    update_cache: yes
```

Installing multiple packages at once:

```
multi_package.yml

- name: Install system packages
  apt:
    pkg:
      - curl
      - ca-certificates
      - python3-pip
      - virtualenv
    state: latest
    update_cache: yes
```

Key apt Parameters

Parameter	Purpose
name / pkg	Package name(s) to install
state: latest	Install & upgrade to newest version
state: present	Install only if not already present
state: absent	Remove the package if installed
update_cache: yes	Run 'apt update' before installing

Note: use pkg: with a list to install many packages in a single task — much faster than separate tasks.

Adding External Repositories – The Docker Example

Third-party software (like Docker) lives outside the default Ubuntu repos.
You must add it safely using a 3-step pattern:

Add GPG Key

Verifies that packages really come from the vendor.
Without it, apt will refuse to install from the repo.

Add GPG Key

```
apt_key:  
  url: https://download.docker.com/linux/ubuntu/gpg  
  state: present
```

Add Repository

Tells apt where to find Docker packages.
Points to the correct distro codename (jammy = Ubuntu 22.04).

Add Repository

```
apt_repository:  
  repo: deb https://download.docker.com/  
        linux/ubuntu jammy stable  
  state: present
```

Install Package

Now apt can safely find, verify, and install Docker.
Always run `update_cache: yes` after adding a new repo.

Install Package

```
apt:  
  name: docker-ce  
  state: latest  
  update_cache: yes
```

Capturing Output with register:

register: saves a task's result into a named variable so later tasks can read it.

```
- name: Check Docker version
  command: docker --version
  register: docker_version      # <- result stored here

- name: Display the version
  debug:
    msg: "Docker version: {{ docker_version.stdout }}"
  when: docker_version is defined and docker_version.stdout is defined
```

What's inside a registered variable?

<code>.stdout</code>	Standard output text from the command
----------------------	---------------------------------------

<code>.rc</code>	Return code: 0 = success, non-zero = failure
------------------	--

<code>.failed</code>	Boolean: true if the task reported failure
----------------------	--

<code>.stderr</code>	Error output text (if any)
----------------------	----------------------------

<code>.stdout_lines</code>	stdout split into a list, one item per line
----------------------------	---

<code>.changed</code>	Boolean: true if the task changed something
-----------------------	---

Jinja2 Templating — Dynamic Values in Playbooks

Ansible uses Jinja2, a Python templating language, to inject dynamic data into tasks, messages, and config files at runtime.

`{{ variable_name }}` ← replaced with its value at runtime

Where you'll use `{{ }}`

Print command output

```
msg: "Docker version: {{ docker_version.stdout }}"
```

Conditional install

```
when: ansible_os_family == "Debian"
```

In a config file (.j2)

```
bind-address = {{ db_host }}  
port = {{ db_port }}
```

Loop over a list

```
with_items: "{{ packages_list }}"
```

Conditional Execution with when:

when: prevents tasks from running unless a condition is true — avoiding errors on missing or undefined data.

```
when_example.yml

- name: Show hello world output
  debug:
    msg: "Hello World Output: {{ hello_world_output.stdout }}"
  when: hello_world_output is defined          # guard 1: variable exists
        and hello_world_output.stdout is defined # guard 2: stdout exists
```

Common when: patterns

variable is defined	Only run if the variable was registered/set
variable.rc == 0	Only run if the previous command succeeded
ansible_os_family == "Debian"	Target only Debian/Ubuntu machines
not variable.failed	Only proceed if the previous task didn't fail
variable.stdout length > 0	Only run if there was any output

Error Handling — ignore_errors & failed_when

By default, Ansible STOPS a playbook when any task fails.
You control this with `ignore_errors` and `failed_when`.

ignore_errors

```
- name: Run optional check
  command: some_optional_cmd
  register: result
  ignore_errors: yes # <- keep going!
```

ignore_errors: no (default) — stop the playbook on failure.

ignore_errors: yes — continue even if this task fails.

Use yes sparingly — only for genuinely optional steps where failure is expected and acceptable.

failed_when

```
- name: Run check
  command: check_service.sh
  register: result
  failed_when:
    - result.rc != 0
    - '"error" in result.stdout'
```

failed_when lets you define exactly what counts as failure — based on return code, output text, or any condition.

Use when the tool doesn't return standard exit codes.

Testing & Verification — Never Assume Success

Always verify your installation worked. The playbook uses two checks for Docker:

Test 1 — Check Version

```
- name: Check Docker version
  command: docker --version
  register: docker_version
  ignore_errors: no
```

Why this matters:

Confirms the Docker binary is installed and in the system PATH.

If this task fails, the installation itself failed — investigate before proceeding.

Use register: to capture the output, then display it with debug: and {{ docker_version.stdout }}.

Test 2 — Hello World Container

```
- name: Run hello-world container
  command: docker run hello-world
  register: hw_output
  ignore_errors: no
```

Why this matters:

Full end-to-end test: proves the Docker daemon is running, Docker can pull an image from the internet, and can execute a container successfully.

More Advanced Playbooks

- The playbook in <https://github.com/HumpreyDev/Strathmore-University-Summer-School-on-Digital-Technologies/tree/main> → Go to Labs
- Lab 3.2 MicroK8s Setup Directory
- Lab 3.3 Helm Setup Directory

Bonus Task

- Write clean-up ansible scripts for all the Labs in the summer-school repository
- This cleanup involves removing installed packages and resetting configs
- This task will be done in Groups